

AI Ops Assignment 2

Question 1 – Single-Stage vs. Multi-Stage Docker

1. The measured image size was: $S_{\text{single}} = 502 \text{ MB}$
2. The measured image size was: $S_{\text{multi}} = 137 \text{ MB}$

The percentage reduction is

$$\text{Reduction} = \frac{S_{\text{single}} - S_{\text{multi}}}{S_{\text{single}}} \times 100 = \frac{502 - 137}{502} \times 100 \approx 72.71\%$$

3. The single-stage image uses the larger `python:3.11` base image as its runtime, whereas the multi-stage build uses `python:3.11` only for dependency installation and `python:3.11-slim` for the final runtime. The final image copies only the installed dependencies, `app.py`, and `model.joblib`; the full builder environment, its `/build` workspace, and other build-time installation material are left behind, while `--no-cache-dir` prevents pip's download cache from being included.

Question 2 – Multi-Container Orchestration with Docker Compose

1. required changes are done in `app.py`
2. `docker-compose.yml` is provided in the repo
3. $T_{\text{miss}} = 18.987 \text{ ms}$, $T_{\text{hit}} = 1.178 \text{ ms}$

$$\text{Speedup} = \frac{T_{\text{miss}}}{T_{\text{hit}}} = \frac{18.987}{1.178} \approx 16.12 \times$$
$$\text{Reduction} = \frac{T_{\text{miss}} - T_{\text{hit}}}{T_{\text{miss}}} \times 100 = \frac{18.987 - 1.178}{18.987} \times 100 \approx 93.80\%$$

4. Docker Compose provides orchestration for multiple interconnected containers as a single application stack on one host. Kubernetes is a cluster-scale container orchestration platform designed to manage workloads across a cluster and provides features such as self-healing and rolling updates.

Question 3 — Kubernetes Indexed Job: Parallel Data Validation

1. The cluster specified in the question has 2 nodes with 2 allocatable CPUs each, giving a total of $2 \times 2 = 4 \text{ CPUs}$.

Each validation pod requests and limits 1 CPU. Therefore, choosing `parallelism = 4`

allows four validation pods to execute concurrently while matching the stated CPU capacity: $4 \text{ pods} \times 1 \text{ CPU per pod} = 4 \text{ CPUs}$.

2. screenshot was provided which shows 4 shvads running at a time
3. The collected results were:

Index	Shard	Invalid Rows	Node
0	0	1	minikube-m02
1	1	2	minikube-m02
2	2	0	minikube
3	3	3	minikube
4	4	1	minikube-m02
5	5	2	minikube-m02
6	6	0	minikube
7	7	2	minikube

The results were obtained directly from pod logs using the Kubernetes API. A shared volume was not used because the workload only needs to report small per-shard results, Using logs avoids introducing shared-volume management and coordination when no persistent shared output is required.

4. If the cluster had 3 nodes with 2 CPUs each, the total available CPU capacity would be $3 \times 2 = 6$ CPUs.

With each validation pod requesting 1 CPU, I would change the Job parallelism to 6 so that up to six shard validations could execute concurrently and use the available CPU capacity more fully.

Question 4 – Kubernetes Deployment: Self-Healing & Rolling Updates

1. The Deployment was successfully applied and reached:

$\text{READY} = 2/2, \quad \text{UP-TO-DATE} = 2, \quad \text{AVAILABLE} = 2.$

2. The ReplicaSet controller, managed by the Deployment, is responsible for maintaining the desired number of pod replicas. It continuously compares the desired state specified by the Deployment with the actual number of matching pods and creates a replacement when the actual count falls below the desired count.
3. A small visible change was made to the API by changing the `/healthz` response from version `v1` to version `v2`.

If this application had been deployed as a Job instead, it would have been treated as a run-to-completion workload rather than a continuously serving application, so the Deployment-style rolling replacement would not be the normal update mechanism.

4. The spam-detection API is a long-running serving workload, so a Deployment is appropriate because it maintains a desired number of ready replicas, automatically replaces failed pods, and supports rolling updates. The shard-validation workload from Question 3 is a finite batch workload, so an Indexed Job is appropriate because each indexed pod processes one shard and terminates after completing its task.

NOTE

All required proofs/evidences are provided in the 'screenshots' folder